

Multi Objective Optimization

```
!pip install pymoo
```

[Show hidden output](#)

Class Development

```
import numpy as np
from pymoo.core.problem import ElementwiseProblem
```

```
class MyProblem(ElementwiseProblem):
    def __init__(self):
        super().__init__(n_var = 3,
                          n_obj = 2,
                          n_ieq_constr = 2,
                          xl = np.array([-10,-10,-10]),
                          xu = np.array([10,10,10]))

    # _evaluate needs to be a method of MyProblem, not nested within __init__
    def _evaluate(self, x, out, *args, **kwargs):
        # objective f1 -> min; f2 -> max
        f1 = x[0]**2 + x[1]**2 + x[2]**2
        # unify the objective by min f2
        # why? Either only min or max can be done
        f2 = (x[0]-1)**2 + (x[1]-1)**2 + (x[2]-1)**2

        # constraints
        g1 = x[0] + x[1] - x[2] - 1
        g2 = -3*x[0] + x[1] + x[2] - 4

        # output
        out["F"] = [f1, f2]
        out["G"] = [g1, g2]

problem = MyProblem()
```

Initializing the Algorithm

```
# Import NSGA2 for multi-objective optimization
from pymoo.algorithms.moo.nsga2 import NSGA2
# Random sampling operator
from pymoo.operators.sampling.rnd import FloatRandomSampling
# simulated binary cross over (SBX) operator
from pymoo.operators.crossover.sbx import SBX
# Polynomial mutation operator
from pymoo.operators.mutation.pm import PM
```

```
algorithm = NSGA2(
    pop_size = 50, # no of solution
    n_offsprings = 10, # solution in generation
    sampling = FloatRandomSampling(),
    crossover = SBX(prob = 0.9, eta = 20),
    # prob -> probability of applying crossover
    # eta -> It controls the spread of the crossover
    mutation = PM(eta = 25),
    # higher eta value; lower mutation rate
    eliminate_duplicates = True
)
```

```
from pymoo.termination import get_termination
# termination criterion
termination = get_termination("n_gen",100)
```

Optimization Process

```
from pymoo.optimize import minimize

res = minimize(
    problem,
    algorithm,
    termination,
    seed = 7, # ensure reproducibility of result
    save_history = True,
    verbose = True
)
```

```
=====
n_gen | n_eval | n_nds | cv_min | cv_avg | eps | indicator
=====
```

1	50	2	0.000000E+00	1.185926E+01	-	-
2	60	3	0.000000E+00	7.6917591249	0.3019672010	ideal
3	70	3	0.000000E+00	5.4148131867	0.000000E+00	f
4	80	3	0.000000E+00	3.7541117350	0.000000E+00	f
5	90	3	0.000000E+00	2.3489280847	0.1318169149	ideal
6	100	3	0.000000E+00	1.0663667524	0.000000E+00	f
7	110	1	0.000000E+00	0.4221170440	9.6024086656	ideal
8	120	1	0.000000E+00	0.1121474931	0.000000E+00	f
9	130	1	0.000000E+00	0.000000E+00	0.000000E+00	f
10	140	1	0.000000E+00	0.000000E+00	0.000000E+00	f
11	150	1	0.000000E+00	0.000000E+00	0.000000E+00	f
12	160	2	0.000000E+00	0.000000E+00	1.0000000000	ideal
13	170	1	0.000000E+00	0.000000E+00	5.8130280678	ideal
14	180	1	0.000000E+00	0.000000E+00	0.000000E+00	f
15	190	1	0.000000E+00	0.000000E+00	1.112144E+01	ideal
16	200	1	0.000000E+00	0.000000E+00	0.000000E+00	f
17	210	1	0.000000E+00	0.000000E+00	0.000000E+00	f
18	220	1	0.000000E+00	0.000000E+00	0.000000E+00	f
19	230	1	0.000000E+00	0.000000E+00	1.3147755098	ideal
20	240	1	0.000000E+00	0.000000E+00	0.000000E+00	f
21	250	1	0.000000E+00	0.000000E+00	0.000000E+00	f
22	260	1	0.000000E+00	0.000000E+00	0.000000E+00	f
23	270	1	0.000000E+00	0.000000E+00	0.000000E+00	f
24	280	2	0.000000E+00	0.000000E+00	1.0000000000	ideal
25	290	2	0.000000E+00	0.000000E+00	0.000000E+00	f
26	300	3	0.000000E+00	0.000000E+00	0.0088822945	f
27	310	4	0.000000E+00	0.000000E+00	0.1980855293	f
28	320	2	0.000000E+00	0.000000E+00	1.1695241566	ideal
29	330	3	0.000000E+00	0.000000E+00	0.2922488991	f
30	340	3	0.000000E+00	0.000000E+00	0.000000E+00	f
31	350	4	0.000000E+00	0.000000E+00	0.0488902423	f
32	360	4	0.000000E+00	0.000000E+00	0.3326102000	ideal
33	370	4	0.000000E+00	0.000000E+00	0.1424254564	f
34	380	4	0.000000E+00	0.000000E+00	0.000000E+00	f
35	390	4	0.000000E+00	0.000000E+00	0.0209795386	ideal
36	400	4	0.000000E+00	0.000000E+00	0.1426878972	ideal
37	410	4	0.000000E+00	0.000000E+00	0.2918922198	ideal
38	420	3	0.000000E+00	0.000000E+00	0.0923657545	ideal
39	430	3	0.000000E+00	0.000000E+00	0.000000E+00	f
40	440	3	0.000000E+00	0.000000E+00	0.0180795975	ideal
41	450	3	0.000000E+00	0.000000E+00	0.000000E+00	f
42	460	3	0.000000E+00	0.000000E+00	0.4124616817	ideal
43	470	3	0.000000E+00	0.000000E+00	0.1149278457	f
44	480	3	0.000000E+00	0.000000E+00	0.000000E+00	f
45	490	5	0.000000E+00	0.000000E+00	0.0038787298	ideal
46	500	6	0.000000E+00	0.000000E+00	0.0680664812	ideal
47	510	7	0.000000E+00	0.000000E+00	0.1361616622	ideal
48	520	8	0.000000E+00	0.000000E+00	0.2556859983	ideal
49	530	6	0.000000E+00	0.000000E+00	0.0044347536	f
50	540	6	0.000000E+00	0.000000E+00	0.0411556552	ideal
51	550	6	0.000000E+00	0.000000E+00	0.0241861518	ideal
52	560	7	0.000000E+00	0.000000E+00	0.0663107715	f
53	570	8	0.000000E+00	0.000000E+00	0.0168220722	f
54	580	9	0.000000E+00	0.000000E+00	0.0018928470	f
55	590	9	0.000000E+00	0.000000E+00	0.0018928470	f

Results and Visualization

```
# res.X -> X rep decision variable vector
```

```
X = res.X
```

```
print(X)
```

```
[ [ 0.8097098 0.93952756 0.91245475]
[ 0.87984285 0.66296041 0.91558947]
[ 0.59521099 0.79305543 0.8780099 ]
[ 0.59521099 0.79385755 0.8780099 ]
[ 0.57435602 0.72709495 0.66265466]
[ 0.57142327 0.72698955 0.84990638]
[ 0.81264254 0.93963295 0.91245475]
[ 0.42067082 0.72709495 0.66081306]
[ 0.87984285 0.66296041 0.93583006]
[ 0.88753129 0.81859364 0.8777483 ]
[ 0.81264254 0.93963295 0.97589884]
[ 0.31710714 0.42926923 0.22188244]
[ 0.23113495 0.34195206 0.36274858]
[ 0.59521099 0.79385755 0.90382612]
[-0.01669947 0.09942786 0.25918194]
[ 0.23113495 0.34195206 0.35418885]
[ 0.81355965 0.93670684 0.97589884]
[ 0.42067082 0.35559723 0.66081306]
[ 0.42067082 0.48497499 0.66081306]
[ 0.82240784 0.48410802 0.60877958]
[-0.01669947 0.09438966 0.24084861]
[ 0.57435602 0.4856422 0.66265466]
[ 0.6015141 0.49036208 0.25918194]
[ 0.59488217 0.86094446 0.97589884]
[ 0.65085385 0.85017376 0.61813829]
[ 0.22126446 0.34195206 0.35418885]
[ 0.59118117 0.84551258 0.88856814]
[ 0.81355965 0.919889 0.97589884]
[ 0.59488217 0.86094446 0.96813475]
[ 0.42217564 0.29643705 0.60117997]
[ 0.59488217 0.48552014 0.54011586]
[ 0.61818077 0.78883549 0.43417377]
[ 0.12303385 0.31983275 0.28284907]
[ 0.41826586 0.34673038 0.35418885]
[ 0.47159965 0.78883549 0.53914543]
[ 0.59165758 0.31546664 0.30618468]
[ 0.54601611 0.42926923 0.22188244]
[ 0.6015141 0.5058888 0.25918194]
[ 0.31710714 0.4683284 0.22181695]
[ 0.61818077 0.79748954 0.43417377]
[ 0.70021143 0.64777957 0.88362651]
[ 0.15111526 0.15925828 0.24605577]]
```

```
# F rep objective function vector
F = res.F
```

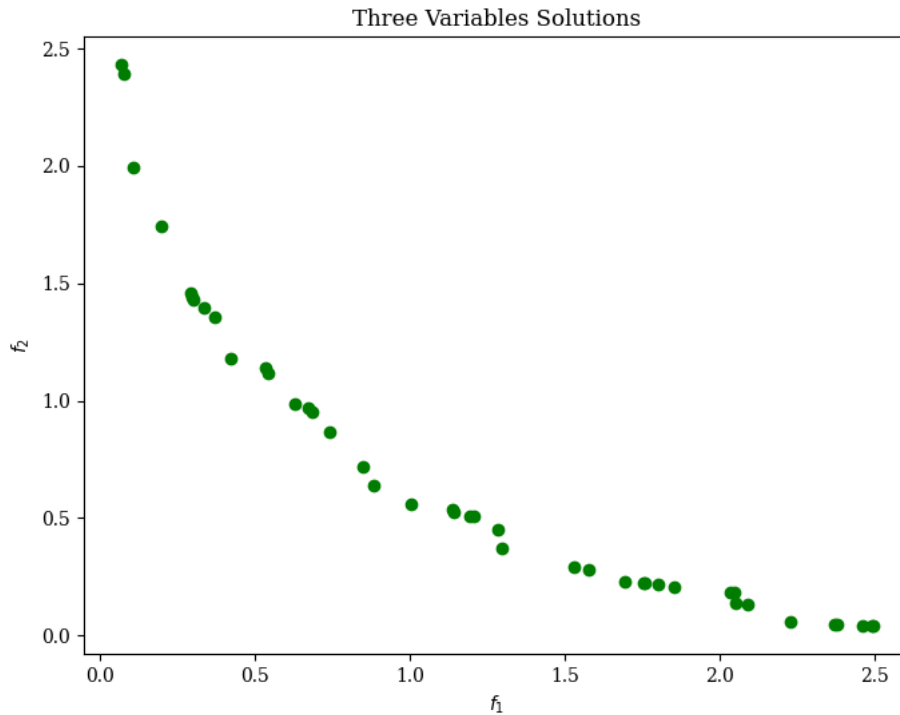
```
print(F)
```

```
[[2.37091566 0.04753145]
[2.05194404 0.13515856]
[1.75411442 0.22156178]
[1.75538731 0.22123044]
[1.29766309 0.36945185]
[1.57737921 0.28074081]
[2.37587167 0.04641117]
[1.1423049 0.52514725]
[2.08941785 0.1321512 ]
[2.22824942 0.06050296]
[2.49567653 0.03932786]
[0.33406083 1.3975432 ]
[0.30194111 1.43026993]
[1.80138759 0.21559827]
[0.07734005 2.3935194 ]
[0.29580432 1.4412526 ]
[2.49167756 0.03934689]
[0.74008723 0.86592501]
[0.84883858 0.71592084]
[1.28132781 0.45073692]
[0.06719633 2.43011874]
[1.00484438 0.55953862]
[0.66944946 0.96733322]
[2.04748869 0.18403777]
[1.5285011 0.2901693 ]
[0.29133891 1.45652818]
[1.85394004 0.20341626]
[2.46045363 0.04175864]
[2.03239504 0.18447229]
[0.62752454 0.98793924]
[0.88133974 0.64030341]
[1.19291575 0.5105357 ]
```

```
[0.19743392 1.74600256]
[0.42061803 1.18224785]
[1.13534546 0.53618431]
[0.54332695 1.11670915]
[0.53163748 1.13730192]
[0.68491796 0.95174829]
[0.36909119 1.3545862 ]
[1.20664388 0.50695574]
[1.69071022 0.22747521]
[0.10874246 1.99588384]]
```

```
from pymoo.visualization.scatter import Scatter
plot = Scatter(title = "Three Variables Solutions")
plot.add(F, color = "green")
plot.show()
```

<pymoo.visualization.scatter.Scatter at 0x7fc2c497d220>



✓ Normalization

```
# In pymoo, minimum of objective vectors as ideal point;
# maximum is nadir point
```

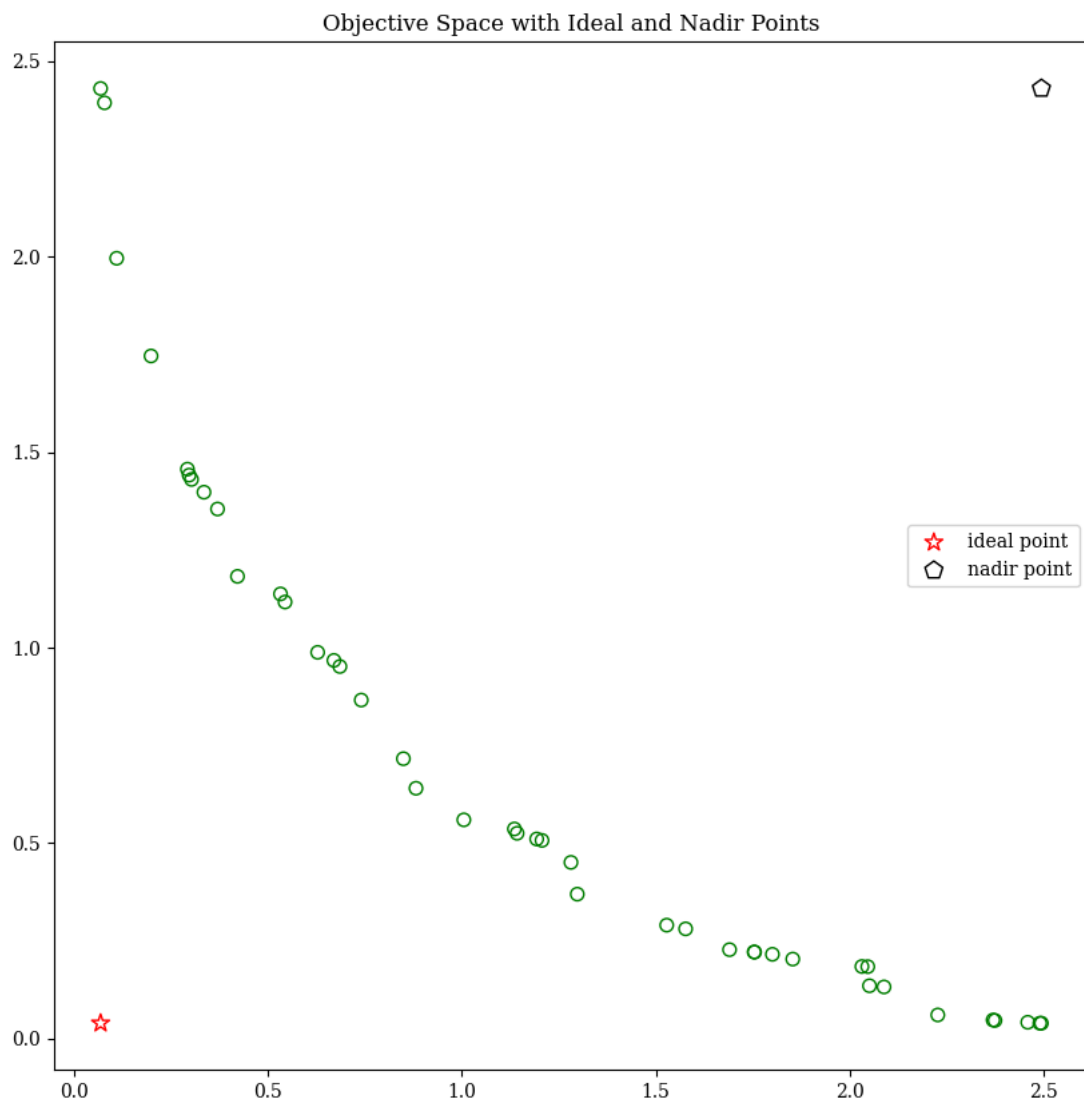
```
ideal_point = F.min(axis=0)
nadir_point = F.max(axis=0)
```

```
print(ideal_point)
print(nadir_point)
```

```
[0.06719633 0.03932786]
[2.49567653 2.43011874]
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(10, 10))
plt.scatter(F[:, 0], F[:, 1], s = 50, facecolor="none", edgecolors="green")
plt.scatter(ideal_point[0], ideal_point[1], facecolor="none",
           edgecolors="red", marker="*", s = 100, label="ideal point")
plt.scatter(nadir_point[0], nadir_point[1], facecolor="none",
           edgecolors="black", marker="p", s = 100, label="nadir point")
plt.title("Objective Space with Ideal and Nadir Points")
plt.legend()
plt.show()
```



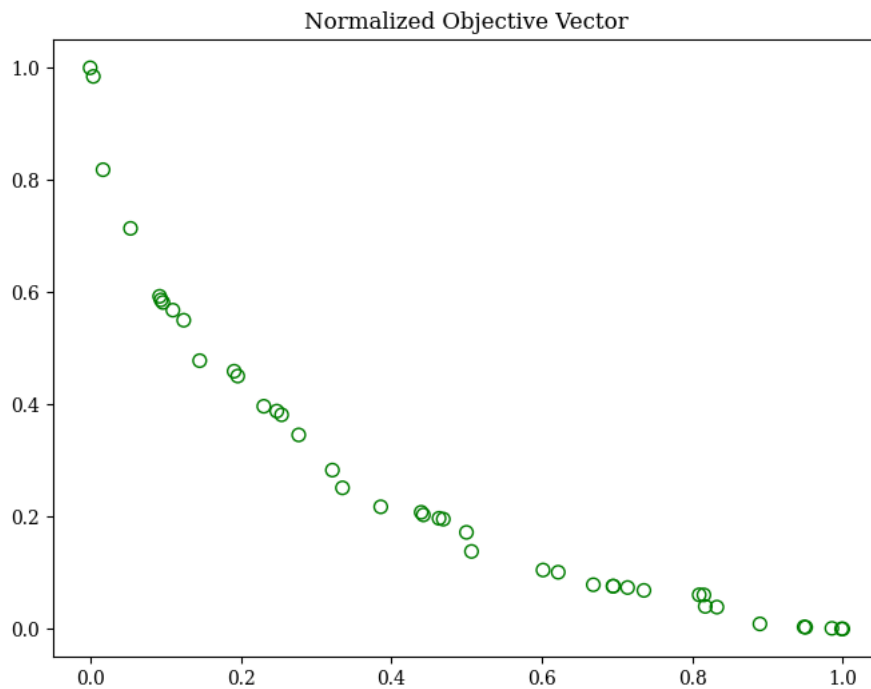
```
# normalized vector
nF = (F - ideal_point) / (nadir_point - ideal_point)
```

```
f1 = nF.min(axis=0)
fu = nF.max(axis=0)
```

```
print(f"Lower bound: {f1}, Upper bound: {fu}")
```

```
Lower bound: [0. 0.], Upper bound: [1. 1.]
```

```
plt.figure(figsize=(8, 6))
plt.scatter(nF[:, 0], nF[:, 1], s = 50, facecolor="none", edgecolors="green")
plt.title("Normalized Objective Vector")
plt.show()
```



Multi-Criteria Decision Making Quantitative Method

Compromise Programming

```
# use the Augmented Scalarization Function (ASF)
from pymoo.decomposition.asf import ASF
decomp = ASF()
```

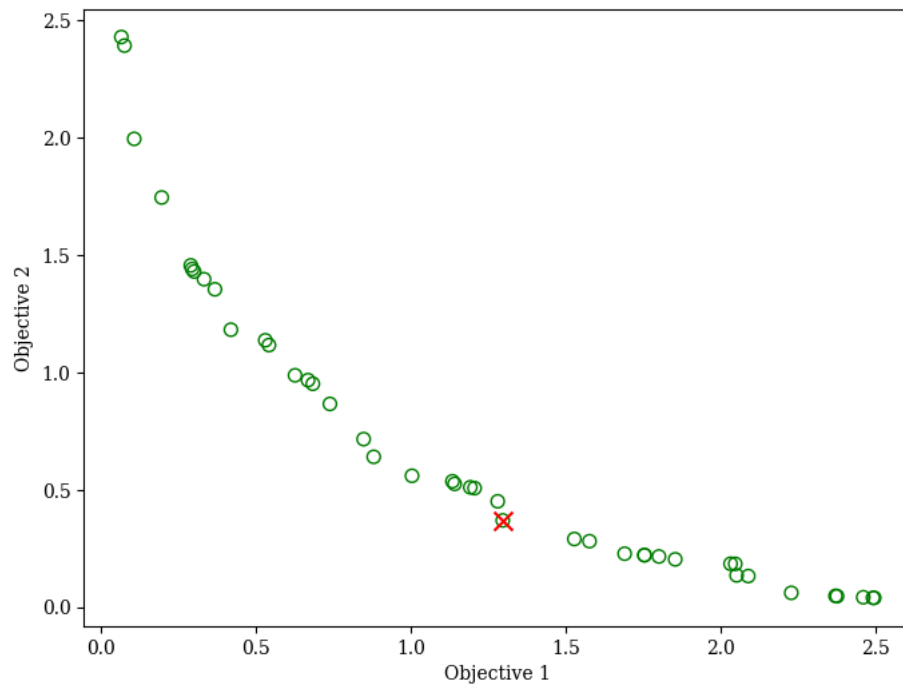
```
Weights = np.array([0.2, 0.8]) # priority with the objective

# recipocal of weight is because of pymoo library decompose
opt_index = decomp.do(nF, 1/Weights).argmin()
```

```
print(f"Best ASF: \nOpt_index = {opt_index}\nF = {F[opt_index]}")
```

```
Best ASF:
Opt_index = 4
F = [1.29766309 0.36945185]
```

```
plt.figure(figsize=(8, 6))
plt.scatter(F[:, 0], F[:, 1], s = 50,
            facecolor="none", edgecolors="green")
plt.scatter(F[opt_index,0], F[opt_index,1], marker = "x",
            color='red', s = 100)
plt.xlabel("Objective 1")
plt.ylabel("Objective 2")
plt.show()
```



```
# decision variable
print(X)
```

```
[ [ 0.8097098  0.93952756  0.91245475 ]
 [ 0.87984285  0.66296041  0.91558947 ]
 [ 0.59521099  0.79305543  0.8780099 ]
 [ 0.59521099  0.79385755  0.8780099 ]
 [ 0.57435602  0.72709495  0.66265466 ]
 [ 0.57142327  0.72698955  0.84990638 ]
 [ 0.81264254  0.93963295  0.91245475 ]
 [ 0.42067082  0.72709495  0.66081306 ]
 [ 0.87984285  0.66296041  0.93583006 ]
 [ 0.88753129  0.81859364  0.8777483 ]
 [ 0.81264254  0.93963295  0.97589884 ]
 [ 0.31710714  0.42926923  0.22188244 ]
 [ 0.23113495  0.34195206  0.36274858 ]
 [ 0.59521099  0.79385755  0.90382612 ]
 [ -0.01669947  0.09942786  0.25918194 ]
 [ 0.23113495  0.34195206  0.35418885 ]
 [ 0.81355965  0.93670684  0.97589884 ]
 [ 0.42067082  0.35559723  0.66081306 ]
 [ 0.42067082  0.48497499  0.66081306 ]
 [ 0.82240784  0.48410802  0.60877958 ]
 [ -0.01669947  0.09438966  0.24084861 ]
 [ 0.57435602  0.4856422  0.66265466 ]
 [ 0.6015141  0.49036208  0.25918194 ]
 [ 0.59488217  0.86094446  0.97589884 ]
 [ 0.65085385  0.85017376  0.61813829 ]
 [ 0.22126446  0.34195206  0.35418885 ]
 [ 0.59118117  0.84551258  0.88856814 ]
 [ 0.81355965  0.919889  0.97589884 ]
 [ 0.59488217  0.86094446  0.96813475 ]
 [ 0.42217564  0.29643705  0.60117997 ]
 [ 0.59488217  0.48552014  0.54011586 ]
 [ 0.61818077  0.78883549  0.43417377 ]
 [ 0.12303385  0.31983275  0.28284907 ]
 [ 0.41826586  0.34673038  0.35418885 ]
 [ 0.47159965  0.78883549  0.53914543 ]
 [ 0.59165758  0.31546664  0.30618468 ]
 [ 0.54601611  0.42926923  0.22188244 ]
 [ 0.6015141  0.5058888  0.25918194 ]
 [ 0.31710714  0.4683284  0.22181695 ]
 [ 0.61818077  0.79748954  0.43417377 ]
 [ 0.70021143  0.64777957  0.88362651 ]
 [ 0.15111526  0.15925828  0.24605577 ]]
```

```
X_optimum = X[opt_index, :]
print(f"x1 = {X_optimum[0]}\nx2 = {X_optimum[1]}\nx3 = {X_optimum[2]}")
```

```
x1 = 0.5743560154459404
x2 = 0.7270949459796138
```

```
x3 = 0.6626546609673184
```

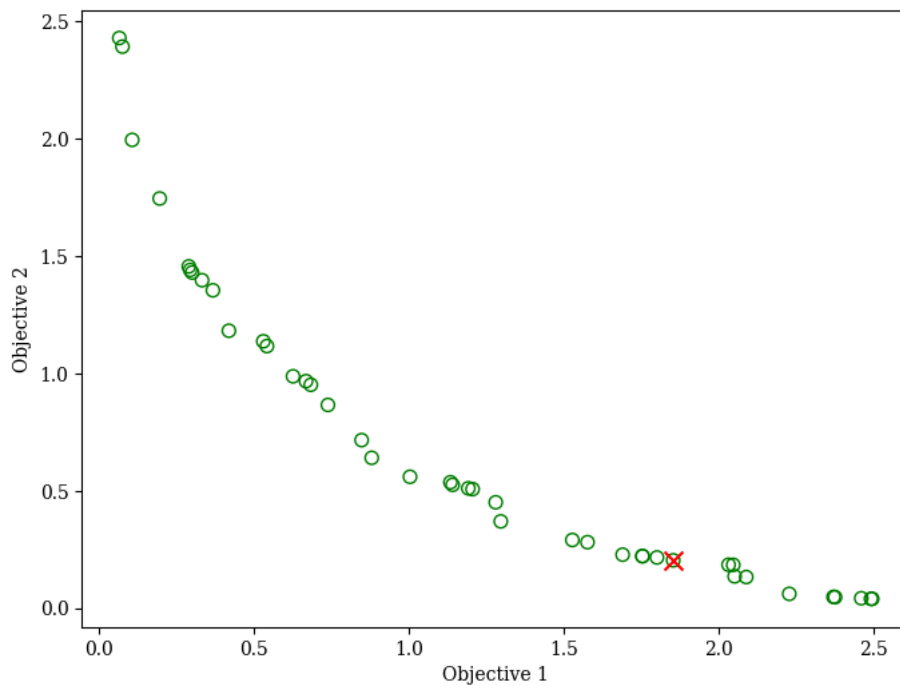
▼ Pseudo Weights

```
from pymoo.mcdm.pseudo_weights import PseudoWeights
# optimal index
Weights = np.array([0.2, 0.8])
opt_idx = PseudoWeights(weights=Weights).do(nF)
```

```
print(f"Best Pseudo weight: \nOpt_index = {opt_idx}\nF = {F[opt_idx]}")
```

```
Best Pseudo weight:
Opt_index = 26
F = [1.85394004 0.20341626]
```

```
plt.figure(figsize=(8, 6))
plt.scatter(F[:, 0], F[:, 1], s = 50,
            facecolor="none", edgecolors="green")
plt.scatter(F[opt_idx,0], F[opt_idx,1], marker = "x",
            color='red', s = 100)
plt.xlabel("Objective 1")
plt.ylabel("Objective 2")
plt.show()
```



```
# decision variable
print(X)
```

```
[ [ 0.8097098  0.93952756 0.91245475]
  [ 0.87984285 0.66296041 0.91558947]
  [ 0.59521099 0.79305543 0.8780099 ]
  [ 0.59521099 0.79385755 0.8780099 ]
  [ 0.57435602 0.72709495 0.66265466]
  [ 0.57142327 0.72698955 0.84990638]
  [ 0.81264254 0.93963295 0.91245475]
  [ 0.42067082 0.72709495 0.66081306]
  [ 0.87984285 0.66296041 0.93583006]
  [ 0.88753129 0.81859364 0.8777483 ]
  [ 0.81264254 0.93963295 0.97589884]
  [ 0.31710714 0.42926923 0.22188244]
  [ 0.23113495 0.34195206 0.36274858]
  [ 0.59521099 0.79385755 0.90382612]
  [-0.01669947 0.09942786 0.25918194]
  [ 0.23113495 0.34195206 0.35418885]
  [ 0.81355965 0.93670684 0.97589884]
  [ 0.42067082 0.35559723 0.66081306]
  [ 0.42067082 0.48497499 0.66081306]
  [ 0.82240784 0.48410802 0.60877958]
```



```
[ -0.01669947  0.09438966  0.24084861]
[  0.57435602  0.4856422   0.66265466]
[  0.6015141   0.49036208  0.25918194]
[  0.59488217  0.86094446  0.97589884]
[  0.65085385  0.85017376  0.61813829]
[  0.22126446  0.34195206  0.35418885]
[  0.59118117  0.84551258  0.88856814]
[  0.81355965  0.919889   0.97589884]
[  0.59488217  0.86094446  0.96813475]
[  0.42217564  0.29643705  0.60117997]
[  0.59488217  0.48552014  0.54011586]
[  0.61818077  0.78883549  0.43417377]
[  0.12303385  0.31983275  0.28284907]
[  0.41826586  0.34673038  0.35418885]
[  0.47159965  0.78883549  0.53914543]
[  0.59165758  0.31546664  0.30618468]
[  0.54601611  0.40036033  0.33180311]
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.